R26-IT-049

Vision-Based Engine Component Identification for an AR-Based Automobile Repair Learning System

Senevirathne S.D., Leelananda M.P.D.R.J., Jayalath S.P.S.C.K., Weerakoon Y.P.B.
Department of Information Technology, Sri Lanka Institute of Information Technology (SLIIT)

Introduction

Automotive vocational education suffers from a knowledge–action gap: learners struggle to correctly identify engine components, leading to frequent misidentifications during repair tasks. This raises safety risks, makes training inefficient and prolongs learning curves. Augmented Reality (AR) can bridge theory and practice by overlaying digital guidance on the real environment in real time [1]. This study introduces an AR-based learning system for the Suzuki Alto 800L that provides real-time, vision-based component recognition for safe, hands-on learning.

Objectives

① Develop marker-less, vision-based detection of 7 Alto 800L engine components using YOLOv11-small.
② Integrate MediaPipe fingertip tracking for natural “point-to-identify” interaction.
③ Implement AR verification with safety-aware feedback for on-device smartphone learning.

Methodology

A custom dataset of 5,150 annotated images of 7 engine components, including the air filter, spark plug wires, battery, oil dipstick, coolant reservoir and high-voltage areas, was collected under real workshop conditions. The data were split 80/20 for training and validation, with augmentation simulating low light, grease, dust and varied illumination. A YOLOv11-small detector [2] was optimised and converted to TensorFlow Lite for on-device use, combined with MediaPipe [3] for fingertip tracking, ARCore [4] for spatial AR overlay, and SQLite for local session logging.

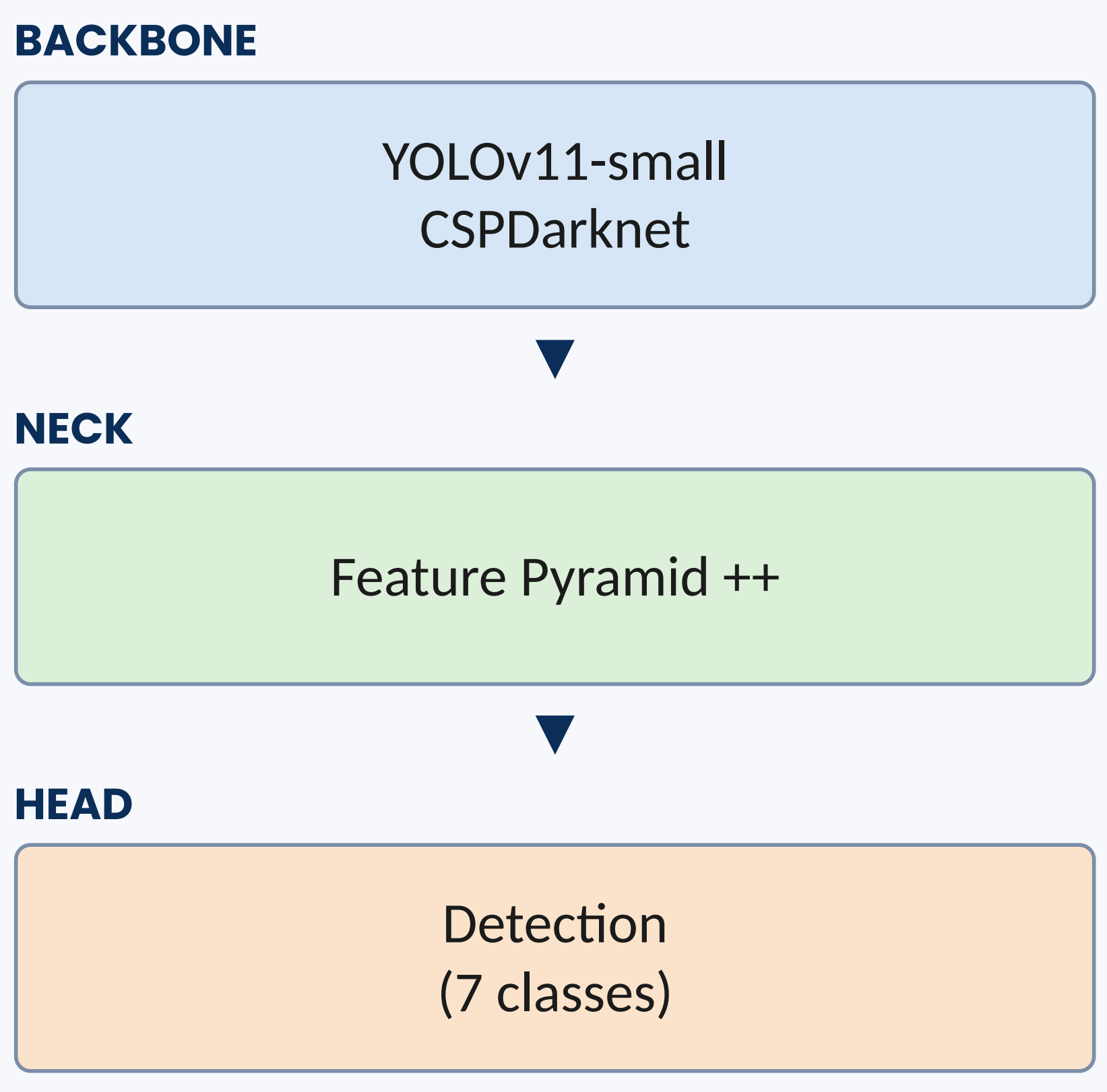

Figure 1. YOLOv11-small detection model: backbone for feature extraction, neck for multi-scale fusion, head for 7-class prediction.

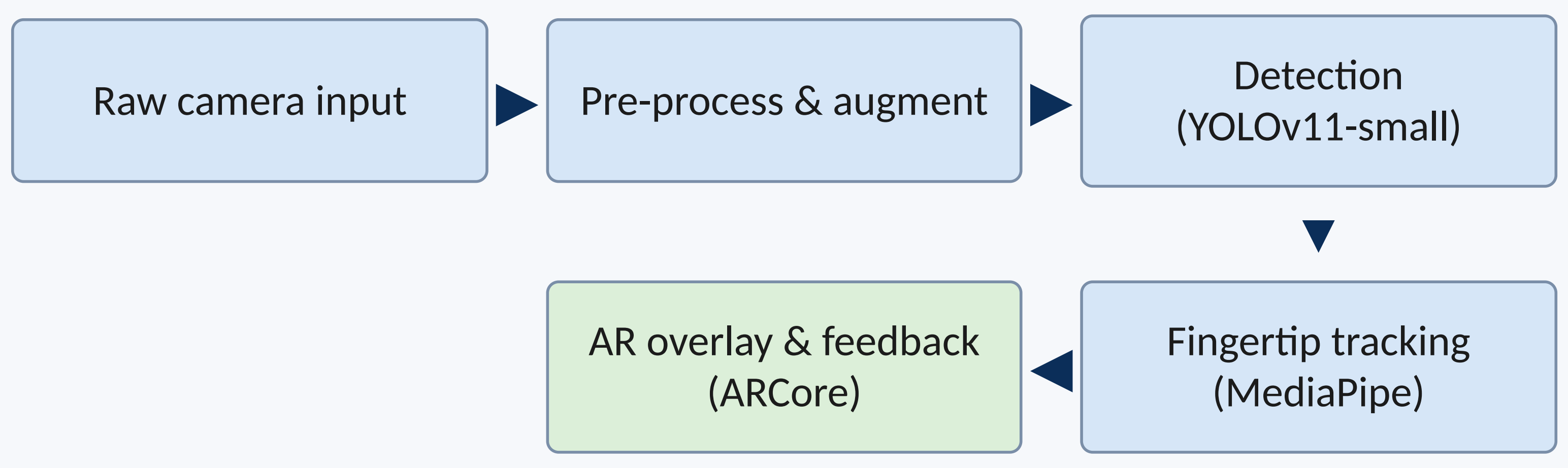

Figure 2. On-device processing pipeline: camera frames are pre-processed, components detected, the user's fingertip matched to a component, and AR feedback shown.

Results

The YOLOv11-small model achieved **85.6% mAP** on the validation set across the 7 components. Fingertip detection reached **85.2%** accuracy with **85.4%** pointing-association accuracy. On a standard Android smartphone the system ran at **148 ms** average latency and **34 FPS**, and identification time fell by **42%** versus manual identification.

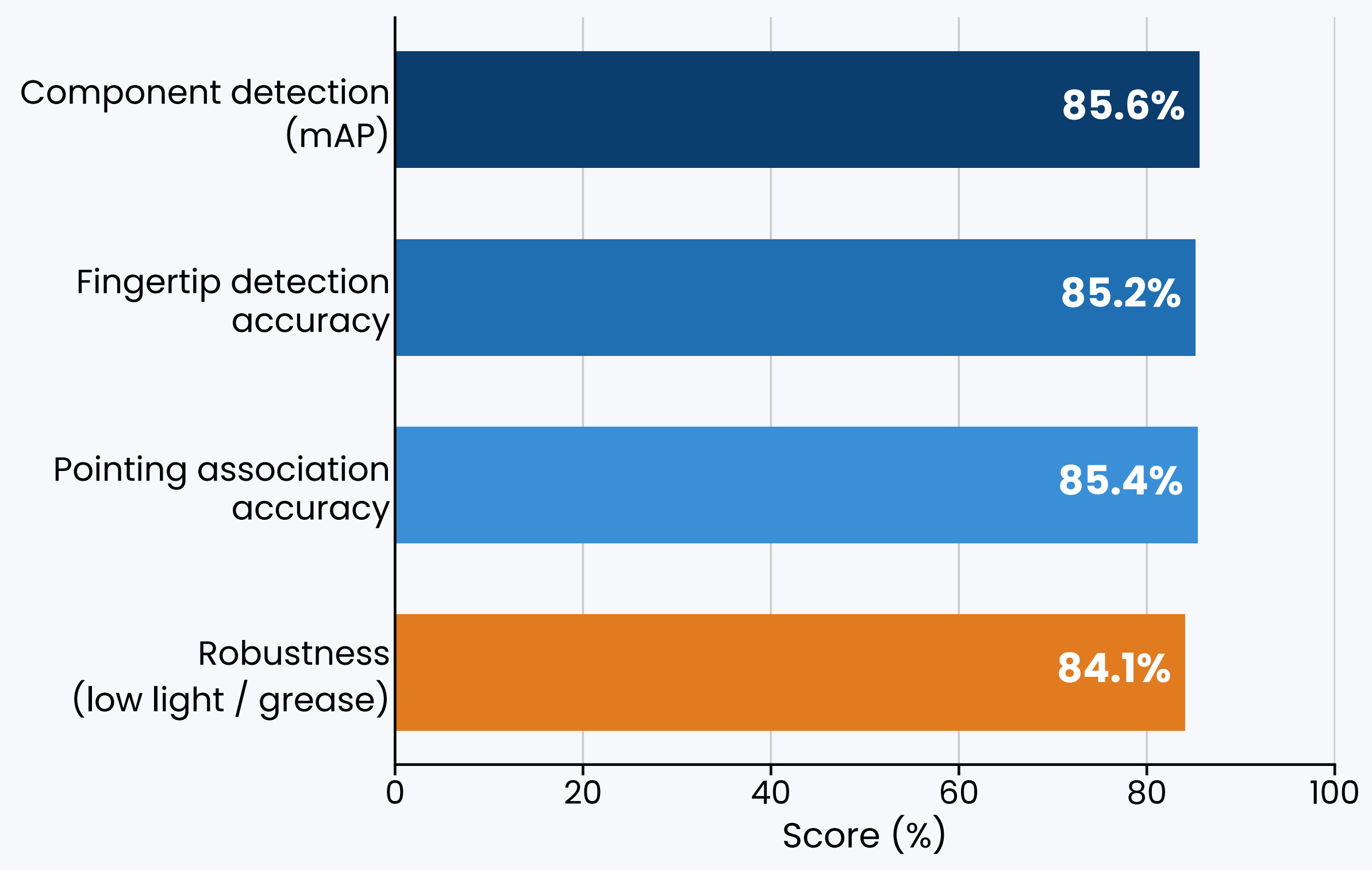

Figure 3. Accuracy of detection, fingertip tracking and pointing association, and robustness under low-light and greasy conditions (%).

Metric	Target	Achieved	Status
mAP (%)	≥ 85	85.6	✔ Met
Latency (ms)	≤ 150	148	✔ Met
Frame rate (FPS)	≥ 30	34	✔ Met
Time reduction (%)	≥ 40	42	✔ Met

Table 1. System performance against design targets on a standard Android smartphone. All four targets were met.

Discussion

All design targets were met, showing that a lightweight detector can run fully on a smartphone in real time. Robustness under low light and grease (84.1%) stayed close to overall mAP, suggesting the workshop-style augmentation was effective. Fingertip and pointing accuracies matched detection accuracy, so point-to-identify is a reliable interaction method. The 42% cut in identification time supports the practical value of the system for vocational training, although latency (148 ms) sits close to its 150 ms limit.

Conclusion

This work demonstrates a lightweight, smartphone-based AR system for the Suzuki Alto 800L that provides marker-less, on-device component identification with privacy-preserving processing, improving training efficiency by 42%. Future work will extend real-time evaluation across diverse devices and add safety alerts for high-voltage components.

References

1. Azuma, R. T. (1997). A survey of augmented reality. *Presence: Teleoperators and Virtual Environments*, *6*(4), 355–385.
2. Jocher, G., & Qiu, J. (2024). *Ultralytics YOLO11* (Version 11.0.0) [Computer software]. https://github.com/ultralytics/ultralytics
3. Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. *arXiv:1906.08172*.
4. Google. (2024). *ARCore overview*. Google for Developers. https://developers.google.com/ar